KHAZANAH NASIONAL BERHAD

Data Associate / AVP — Take-Home Assessment

Candidate Brief

Overview

Time Allowed	48 hours from receipt of this brief
Expected Effort	4–5 hours
Deliverables	Working MVP (code repository) + README + short walkthrough video (3–5 min, e.g. Loom) OR a 3–5 slide deck
Cost	This assessment can be completed entirely using free tools and APIs. You are not expected to spend any money. Spending money to provision services is entirely up to the candidate

Context

Khazanah Nasional Berhad is Malaysia's sovereign wealth fund. Each year, Khazanah publishes an Annual Review, a comprehensive document covering portfolio performance, investment strategy, financial highlights, sustainability initiatives, and more.

Your task is to build an end-to-end MVP that allows users to interactively query and extract insights from Khazanah's Annual Review using AI.

Data Source

Use Khazanah's latest publicly available Annual Review, downloadable from:

https://www.khazanah.com.my/media-downloads/khazanah-annual-review/

Download the most recent Annual Review PDF. This is a public document, no special access is required.

Priority Guide

If you're short on time, focus on what matters most:

Priority	Section	What we care about
Critical	Section 1: Data Ingestion & Pipeline, Section 2: RAG-Powered Query Engine, Section 4: API & User Interface	Can you build a workable (POC) from scratch?
Important	Section 3: Structured Data Extraction	Do you understand the data well and are you able to model it?
Writeup	Section 5: README	Can you communicate to non-engineers?
Required	Section 6: Design Questions	Do you think about production and blast radius?
Stretch	Stretch Goals (Bonus)	Are you able to improve your POC?

Section 1: Data Ingestion & Pipeline

Build a pipeline that:

- Parses the Annual Review PDF, handling its complexity (tables, charts, mixed layouts, footnotes)
- Chunks the document into meaningful segments using a strategy you can justify
- Generates embeddings for each chunk
- Stores the embeddings in a vector database for retrieval

What we're looking for:

Your approach to document parsing (how do you handle tables and structured content?), chunking strategy (naive splitting vs. semantic chunking. Do you think about it?), and your choice of embedding model and vector database with justification.

Section 2: RAG-Powered Query Engine

Build a Retrieval-Augmented Generation (RAG) system that can answer natural language questions about the Annual Review. Users should be able to ask questions such as:

- "What was Khazanah's TWRR as of the latest reporting period?"
- "Which sectors did Khazanah increase or decrease exposure to? (if any)"
- "Summarise Khazanah's sustainability or ESG initiatives."
- "What were the key financial highlights for the year?"

What we're looking for:

RAG architecture decisions (retrieval strategy, prompt engineering, context window management), LLM selection and cost-awareness (why did you pick this model?), response quality and accuracy, source attribution (can the user see where the answer came from?), and how you handle edge cases (ambiguous queries, information not in the document).

Section 3: Structured Data Extraction

Extract structured data from the Annual Review into a clean, standardised format (JSON or table). At minimum, extract:

- Portfolio company names, sectors, and ownership stakes (where available)
- Key financial metrics (e.g., TWRR, total assets, realisable asset value)
- Any other structured data you think is valuable from the report

What we're looking for:

Can you use LLMs for reliable structured extraction (not just chat)? Schema design thinking. Accuracy and handling of inconsistent formats across different sections of the report.

Section 4: API & User Interface

Wrap your solution in:

- A FastAPI backend with documented endpoints (e.g., /query, /extract, /health)
- A simple Streamlit, Gradio, NextJS (or any) frontend where a user can:
    - Ask questions about the Annual Review in natural language
    - View extracted structured data
    - See source citations / references for answers

Your application should be runnable with clear setup instructions. Ideally, it should work with a single command (e.g., docker compose up or a setup script).

What we're looking for:

Full-stack engineering ability — can you ship something usable, not just a notebook? API design (RESTful conventions, input validation, error handling).

Section 5: README

Your README should not be an afterthought. You are required to write it as if you are handing this project to a non-technical stakeholder – for example, a project manager, who needs to understand what this tool does, why it works the way it does, and what its limitations are.

Your README should include (non-exhaustive):

- what the tool does in plain English (not technical jargon)
- an architecture overview (a simple diagram or description of how the components fit together),
- your design decisions and trade-offs (why you chose the tools and models you did, and what you would do differently with more time)
- known limitations (be honest about what doesn't work well)
- setup instructions clear enough for a reviewer to run it locally.

Section 6: Design Questions

Please provide short written answers to the following questions. We want to understand how you think about building systems beyond a POC.

1. If this tool were to be deployed internally at Khazanah for daily use by analysts, what would you change about your current architecture?
2. A user reports that the tool confidently returned an incorrect answer and shared it in a presentation. How would you prevent this from happening?
3. You need to push an update to the RAG pipeline, but the tool is actively being used by 20 analysts. How do you roll out the change safely?

Stretch Goals (Bonus)

These are entirely optional. Completing the core sections well is more important than attempting all stretch goals poorly.

- Evaluation metrics on RAG quality (e.g., using the RAGAS framework)
- A comparison view if multiple years' reports are ingested
- Caching for repeated queries to improve response time
- Guardrails against hallucination (e.g., confidence scoring, refusal when no relevant context is found)
- An agentic layer where the system decides whether to search, extract, or compare based on the query type
- Multi-modal handling (extracting insights from charts/graphs in the PDF, not just text)

Suggested Free Tools & Resources

You are not expected to spend any money on this assessment. Below are some free options for each component. You are free to use any tools you prefer — we care about your choices and reasoning, not which specific provider you pick.

Component	Free Options	Notes
LLM API	Google Gemini (free tier), Groq (free tier), Mistral (free experiment plan), Ollama (local)	Gemini and Groq have the most generous free quotas
Embeddings	HuggingFace sentence-transformers (local), Gemini embedding API, Ollama embeddings	sentence-transformers is fully local with zero API limits
Vector Database	ChromaDB (local), FAISS (local), Pinecone Starter (cloud, free), Qdrant Cloud (free tier)	ChromaDB or FAISS are simplest for local development
PDF Parsing	Unstructured.io, pdfplumber, PyMuPDF, LlamaParse (1,000 pages/day free)	Unstructured.io is best for complex layouts like annual reports
Backend	FastAPI (open-source)	
Frontend	Streamlit, Gradio (both open-source), NextJS	

Submission Instructions

- Push your code to a GitHub repository, set to public, and share the link with us.
- Your repository must include a README.md with (non-exhaustive): (a) setup instructions, (b) architecture overview, (c) your design decisions and trade-offs, and (d) known limitations.
- Include a short walkthrough — either a 3–5 minute Loom/video recording demonstrating your MVP in action, OR a 3–5 slide deck walking through your approach and a live demo screenshot.
- Ensure your application can be run locally by the reviewer following your README instructions.
- Submit within 48 hours of receiving this brief.

Important Notes

- You may use AI coding assistants (e.g., GitHub Copilot, ChatGPT, Claude). We assume you will. What differentiates candidates is architecture decisions, trade-off reasoning, and the judgment calls in your README.
- We value a working MVP with honest documentation over a polished but non-functional submission.
- If you run into issues or make trade-offs due to time constraints, document them. We'd rather see 'I chose X over Y because of Z, and given more time I would do W' than a half-finished attempt at everything.
- Quality over quantity. A well-executed core submission beats a rushed attempt at all stretch goals.

Good luck. We look forward to reviewing your work.